

Universidad Nacional Autónoma de México

Facultad de Ingeniería

División de Ingeniería Eléctrica

Departamento de Ingeniería en computación

Semestre 2027-1



Sistemas operativos

Profesor Gunnar Eyal Wolf Iszaevich

Grupo: 7

Investigación sobre microkernels y exokernels

Alumnos:

Almaraz García Beatriz

Galicía Barragán Edson

Fecha de entrega: 17 de septiembre de 2026

Índice

Índice.....	2
1. Introducción.....	3
2. El núcleo de un sistema operativo.....	3
2.1 ¿Qué es el Kernel (núcleo)?.....	3
2.2 Funciones principales del núcleo.....	3
3. Arquitecturas del núcleo.....	3
3.1 Núcleos monolíticos.....	4
3.2 Micronúcleos.....	4
3.3 Núcleos híbridos.....	4
4. Exokernels.....	5
4.1 ¿Qué es un exokernel?.....	5
4.2 Origen y propósito.....	5
5. Principios fundamentales del diseño de un exokernel.....	5
5.1 Asignación minimalista de recursos.....	5
5.2 Multiplexación segura de recursos.....	6
5.3 Sistemas operativos de biblioteca (LibOS).....	6
6. Ventajas del modelo exokernel.....	7
7. Comparación de las arquitecturas.....	7
8. Conclusiones.....	8
9. Referencias.....	8

1. Introducción

Con la presente investigación, buscamos explicar un poco a profundidad, la diferencia entre 4 organizaciones de un sistema operativo; siendo el microkernel, el monolítico, el híbrido y finalmente un exokernel. Existen varios tipos más, que en este caso no abordaremos, pero inicialmente sirve como una pequeña introducción a la organización de un sistema operativo.

2. El núcleo de un sistema operativo

2.1 ¿Qué es el Kernel (núcleo)?

El núcleo es la capa de software que actúa como intermediario entre el código de la aplicación y el hardware, controlando todas las operaciones privilegiadas que realiza un sistema en ejecución. La arquitectura del núcleo ya sea monolítica, de micronúcleo o híbrida determina dónde se ejecutan los servicios del sistema, cómo se propagan los fallos y qué límites de rendimiento se aplican.

El kernel o núcleo es la parte principal de un sistema operativo que se encarga de controlar y administrar los recursos de la computadora. Funciona como un intermediario entre los programas que utiliza el usuario y el hardware de la computadora.

2.2 Funciones principales del núcleo

El ámbito del kernel abarca la gestión de procesos , la gestión de memoria , la arbitraje del sistema de archivos , la ejecución de controladores de dispositivos , las llamadas al sistema y el funcionamiento de la pila de red . El estándar IEEE POSIX (IEEE Std 1003.1) define la interfaz de llamadas al sistema que utilizan los procesos del espacio de usuario para solicitar servicios del kernel, estableciendo un límite formal entre la ejecución privilegiada y la no privilegiada.

Tres categorías arquitectónicas principales rigen la forma en que los ingenieros de kernel asignan estas responsabilidades: kernels monolíticos, microkernels y kernels híbridos. Una cuarta categoría “exokernels” existe en la literatura de investigación, pero no ha alcanzado una implementación generalizada en producción hasta la década de 2020.

3. Arquitecturas del núcleo.

3.1 Núcleos monolíticos.

En un núcleo monolítico, todos los servicios principales del sistema operativo (planificación, gestión de memoria, sistemas de archivos, controladores de dispositivos y redes) se ejecutan dentro de un único binario de gran tamaño en el espacio del núcleo. Una llamada al sistema desde un proceso del espacio de usuario activa un cambio de contexto al modo núcleo, donde el servicio solicitado se ejecuta directamente sin la sobrecarga del paso de mensajes. El núcleo Linux, gobernado por la comunidad del núcleo Linux y documentado en kernel.org, es el núcleo monolítico de producción estándar. A partir de la versión 6.x del núcleo, el árbol de código fuente de Linux contiene más de 27 millones de líneas de código

3.2 Micronúcleos.

La arquitectura de microkernel reubica todos los servicios no esenciales (controladores de dispositivos, sistemas de archivos, pilas de red y manejadores de protocolos) en procesos de servidor en el espacio de usuario. El kernel en sí conserva solo cuatro mecanismos: gestión del espacio de direcciones, gestión de subprocesos, comunicación entre procesos (IPC) y una primitiva planificación mínima. La familia de microkernels L4, desarrollada por Jochen Liedtke y posteriormente formalizada en el proyecto L4.verify de NICTA (ahora Data61, CSIRO), demostró que un microkernel puede verificarse formalmente para comprobar su corrección, una propiedad inalcanzable con diseños monolíticos de escala de código comparable. El microkernel seL4, cuya verificación formal de corrección funcional se completó en 2009 (publicada en ACM SOSP 2009), contiene menos de 9000 líneas de código C sujetas a verificación.

3.3 Núcleos híbridos

Los núcleos híbridos conservan una estructura monolítica, pero trasladan selectivamente ciertos servicios, generalmente el aislamiento de controladores de dispositivos o los servidores de personalidad al espacio de usuario, u organizan el código del núcleo en módulos cargables con interfaces definidas. El núcleo de Windows NT (documentado en Windows Internals por Russinovich et al., Microsoft Press) utiliza un diseño híbrido: los servicios ejecutivos principales se ejecutan en modo núcleo, mientras que el subsistema Win32 originalmente se ejecutaba parcialmente en modo de usuario antes de ser trasladado completamente al espacio del núcleo por razones de rendimiento en Windows Vista. El núcleo XNU de Apple, que sirve de base a macOS e iOS, combina un núcleo de micronúcleo Mach con una capa monolítica derivada de BSD que se ejecuta en el espacio del núcleo.

4. Exokernels

El término exo-kernel (a veces escrito exo-kernel o exokernel) se refiere a un enfoque radical del diseño de sistemas operativos (SO) que traslada las responsabilidades tradicionales del kernel al espacio de usuario. A diferencia de los kernels monolíticos, que agrupan controladores de dispositivos, lógica del sistema de archivos, pilas de red y muchos otros servicios en un único componente privilegiado, un exo-kernel proporciona sólo las abstracciones mínimas necesarias para la multiplexación segura de recursos. Todas las políticas de nivel superior (estrategias de gestión de memoria, semántica del sistema de archivos, algoritmos de planificación e incluso protocolos de red) se implementan como bibliotecas de nivel de usuario.

Exokernel Approach

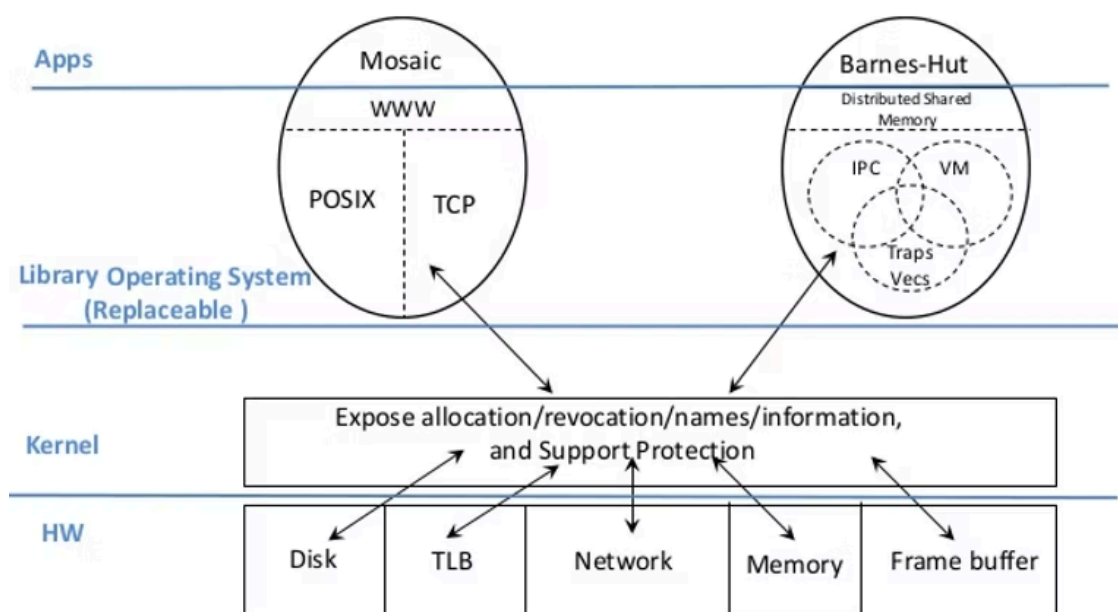


Diagrama de organización de un exokernel

4.1 ¿Qué es un exokernel?

- Busca reducir las responsabilidades tradicionales del kernel.
- Proporciona únicamente mecanismos mínimos.
- Permite que las aplicaciones tengan mayor control sobre los recursos.
- Las políticas se implementan principalmente en el espacio de usuario.

4.2 Origen y propósito

La idea fue planteada por primera vez a finales de la década de 1990 por investigadores (principalmente Dawson R. Engler, M. Frans Kaashoek, James W. O'Toole Jr.) del Instituto Tecnológico de Massachusetts (MIT), siendo como una forma de eliminar la sobrecarga de rendimiento y la inflexibilidad inherentes a los núcleos de los sistemas operativos convencionales.

Los primeros prototipos, fueron dos importantes, uno llamado Aegis y el otro XOK/ExOS. Para el año 1994-1995 fueron las primeras publicaciones y el prototipo Aegis fue dado al conocimiento del mundo. Pero a su vez, Aegis solamente era un prototipo inicial, el cual ayudó a validar la idea de separar la protección y gestión de recursos. Posteriormente, llegó XOK en el año 1997, el cual hizo una implementación completa siendo capaz de ejecutar aplicaciones reales con LibOS como ejemplo ExOS (similar a Unix).

Desde entonces, el concepto de exo-núcleo ha inspirado diversos proyectos de investigación, productos comerciales e incluso plataformas nativas de la nube que buscan brindar a las aplicaciones un control más directo sobre el hardware, preservando al mismo tiempo la seguridad y el aislamiento.

5. Principios fundamentales del diseño de un exokernel

5.1 Asignación minimalista de recursos

Un exo-kernel no proporciona abstracciones de alto nivel como archivos, sockets o procesos. En cambio, solo ofrece las primitivas necesarias para asignar, proteger y revocar recursos de hardware:

- Páginas físicas (o marcos)
- Segmentos de tiempo de CPU
- Puertos de E/S y canales DMA
- Vectores de interrupción

Estas primitivas se exponen a través de tokens de capacidad, que son objetos infalsificables generados por el núcleo que codifican los derechos del titular.

5.2 Multiplexación segura de recursos.

La seguridad se aplica mediante el control de acceso basado en capacidades. Cuando un proceso solicita un recurso, el núcleo verifica sus capacidades existentes y, si está autorizado, devuelve un nuevo token. El token puede ser:

Transferido entre procesos (delegación)

Revocado por el núcleo (por ejemplo, cuando finaliza un proceso).

Verificado por las bibliotecas de espacio de usuario antes de utilizar el hardware subyacente.

Dado que el núcleo nunca interpreta la semántica del recurso, no se le puede engañar para que viole las políticas de aislamiento.

5.3 Sistemas operativos de biblioteca (LibOS)

Todas las políticas; diseño del sistema de archivos, pilas de protocolos de red, estrategias de memoria virtual, se implementan en el espacio de usuario como sistemas operativos de biblioteca (LibOS). Un LibOS típicamente:

- Mapea las páginas físicas sin procesar a su espacio de direcciones virtuales.
- Instala su propio gestor de fallos de página para administrar la paginación bajo demanda.
- Registra los controladores de interrupción para la entrada/salida del dispositivo.
- Proporciona API similares a POSIX a las aplicaciones que se enlazan con él.
- Esta separación genera dos beneficios principales:
- Rendimiento: No se producen transiciones al modo kernel en la mayoría de las operaciones.
- Flexibilidad: Los desarrolladores pueden cambiar una biblioteca de sistema operativo (LibOS) sin reiniciar ni recompilar el kernel.

Qué hace un LibOS:

- Mapear páginas físicas.
- Gestionar fallos de página.
- Manejar interrupciones.
- Proporcionar APIs similares a POSIX.
- Permitir implementar diferentes políticas sin modificar el kernel.

6. Ventajas del modelo exokernel

Rendimiento

- Menos transiciones entre espacio de usuario y kernel.
- Acceso más directo a los recursos.

Flexibilidad

- Las políticas pueden modificarse desde el espacio de usuario.
- Se puede cambiar un LibOS sin modificar el kernel.

Seguridad y aislamiento

- El kernel controla el acceso a los recursos.
- Las capacidades determinan qué recursos puede utilizar cada proceso.

7. Comparación de las arquitecturas

Aspecto	Exokernel	Microkernel	Monolítico	Híbrido
Definición	Núcleo mínimo que multiplexa y protege recursos; deja abstracciones a librerías en espacio usuario	Núcleo pequeño que sólo implementa funciones básicas (IPC, planificación, memoria); servicios en espacio usuario	Kernel grande que incluye la mayoría de servicios del sistema (drivers, FS, red) en espacio kernel	Combina partes monolíticas y microkernel: servicios críticos en kernel, otros en espacio usuario
Funciones en kernel	Multiplexado y protección de recursos	Gestión mínima: IPC, scheduling, protección de memoria	Gestión completa: procesos, memoria, I/O, sistemas de archivos, drivers	Núcleo con servicios esenciales; módulos adicionales en espacio usuario o módulos cargables
Rendimiento	Muy alto para aplicaciones que controlan recursos; bajo overhead de abstracción	Puede tener overhead por IPC entre procesos de usuario y kernel	Alto rendimiento en llamadas internas; menor overhead por comunicación	Intermedio: busca equilibrio entre rendimiento y modularidad
Flexibilidad y personalización	Muy alta; aplicaciones pueden implementar sus propias abstracciones	Alta; servicios pueden reemplazarse sin tocar kernel	Baja; cambios requieren modificar kernel	Alta; permite extensiones y modularidad sin sacrificar todo el rendimiento
Seguridad y almacenamiento	Fuerte aislamiento por diseño; control fino de recursos	Muy buena: menor superficie de ataque en kernel	Mayor superficie de ataque; fallos en drivers afectan todo el sistema	Mejor que monolítico; depende de qué se mantenga en kernel
Complejidad de desarrollo	Alta para desarrolladores de	Moderada-alta: diseño de IPC y	Alta en el kernel; pero más sencillo	Alta: combina complejidades de

	LibOS; requiere más trabajo en espacio usuario	servicios distribuidos	para programadores de usuario	ambos enfoques
Escalabilidad en sistemas distribuidos/ nube	Buena para control fino de recursos en entornos especializados	Buena si se diseña para microservicios y aislamiento	Buena en servidores optimizados; depende de modularidad	Buena: permite optimizar rutas críticas en kernel y mover lo demás fuera
Ejemplos	Investigación (MIT Xok/Aegis), sistemas experimentales de alto rendimiento	Sistemas embebidos, RTOS, seL4, L4, QNX, Minix	Linux (kernel monolítico con módulos)	Windows NT (modelo híbrido), macOS (componentes híbridos)

8. Conclusiones

Con la propia investigación, pudimos llegar a una conclusión, y es que, la arquitectura de un exokernel, puede que sea muy poco viable para el uso cotidiano por la especificidad que tiene que contener el diseño propio de dicha arquitectura. Pero en otros entornos como la investigación, aplicaciones como Firecracker (más actual que el proyecto del MIT), o gVisor, ambos proyectos de software libre, que están en github.

También podemos decir que, el exokernel es bastante parecido al microkernel, ya que ambos reducen el kernel y delegan funciones, mientras que el kernel monolítico se contrastan totalmente ya que uno es un núcleo grande y el otro es pequeño, y en el caso de un híbrido, no se parecen ya que el híbrido busca equilibrio mientras los exokernels no les gusta la abstracción y son extremos minimalistas

9. Referencias

- 1.2.1 Tipos: monolítico, por capas, micro núcleo (microkernels), máquina virtual y exokernels. (n.d.). https://www.comunidad.escom.ipn.mx/israelst/RDD_SO_U1/pages/1-2-1-tipos-monol%C3%ADtico-por-capas-micro-n%C3%BAcleo-microkernels-m%C3%A1quina-virtual-y-exokernels.html
- Firecracker. (n.d.). <https://firecracker-microvm.github.io/>
- GeeksforGeeks. (2025, July 23). Exokernel in OS. GeeksforGeeks. Recuperado el 17 de septiembre de 2026. <https://www.geeksforgeeks.org/operating-systems/exokernel-in-os/>

- Internal structure of XOK/EXOS. (n.d.).
<https://pdos.csail.mit.edu/archive/exo/exo-internals/internals.html>
- Introduction to gVisor security - gVisor. (n.d.).
https://gvisor.dev/docs/architecture_guide/intro/
- Martinuke. (2026, April 1). Understanding the Exo-Kernel: architecture, benefits, and Real-World Applications. martinuke0's Blog. Recuperado el 14 de septiembre de 2026.
<https://martinuke0.github.io/posts/2026-04-01-understanding-the-exokernel-architecture-benefits-and-realworld-applications/>
- Md. Ratan Rana, Saikat Baul, "An Overview of Operating Systems Based on Microkernel Technology and their Essential Components", International Journal of Information Engineering and Electronic Business(IJIEEB), Vol.16, No.6, pp. 18-28, 2024. DOI:10.5815/ijieeb.2024.06.02.
https://www.researchgate.net/profile/Saikat-Baul/publication/386549964_An_Overview_of_Operating_Systems_Based_onMicrokernel_Technology_and_their_EssentialComponents/links/675777ab10214e7cc4a842af/An-Overview-of-Operating-Systems-Based-onMicrokernel-Technology-and-their-EssentialComponents.pdf
- MIT Exokernel Operating System. (marzo, 1998).
<https://pdos.csail.mit.edu/archive/exo/>
- Operating Systems Authority. (2026, March 10). The operating system kernel: monolithic, microkernel, and hybrid designs. Recuperado el 14 de septiembre de 2026. <https://operatingsystemsauthority.com/operating-system-kernel.html>
- Sugathadasa, K. M. (2017, September 1). An introduction on Exokernel operating Systems. keetmalin. Recuperado el 17 de septiembre de 2026
<https://keetmalin.wixsite.com/keetmalin/post/2017/09/01/an-introduction-on-exokernel-operating-systems>